

Optimus-Cost-Agent

Enterprise Architectural Blueprint

Adaptive Agent Reasoning Swarms and Evolutionary Governance Contracts

Version 2.17

Architected by: Vibhanshu Agarwal

1. Executive Summary

Conceptual framing adapted from evolutionary software boundaries formulated by Neal Ford, Rebecca Parsons, and Patrick Kua (O'Reilly, 2026). The Optimus-Cost-Agent v2.17 transitions from a basic token-routing utility to a formal Harness Engineering framework designed to govern non-deterministic generation via deterministic software boundaries. Standard AI agents are fundamentally bounded by the Dreyfus model's skill acquisition ceiling; they cannot brute-force their way past the competent or proficient barrier into true architectural consistency through raw scaling alone. Optimus raises the agent's effective architectural competence through deterministic constraints and feedback by wrapping logical model abstraction inside an opinionated local-first Python ACP server governed by automated, multi-tier architectural fitness functions and explicit request-level cost attribution.

2. Harness Engineering & Context Optimization

To protect the token window from context bloat while systematically mitigating hallucination, the agent structure enforces a strict division between Feedforward Controls and Feedback Sensors:

- **Feedforward Controls (Pre-Generation):** Before a prompt payload hits the downstream model, the Context Optimization Node constructs structural boundaries. It strips code down to AST-based functional slices, structures static filesystem metadata layouts at the head of the prompt window to maximise GLM-5.2 input caching, and injects succinct, unambiguous constraints using an Architecture Definition Language (ADL) dialect to minimise context attention allocation.
- **Feedback Sensors (Post-Generation Evaluation):** Once the primary model provider emits a candidate patch, the engine catches the text stream inside a validation loop. It intercepts execution blocks and evaluates them via automated, triggered fitness functions before any source mutations are applied to the active working tree.

3. Automated Architectural Fitness Functions

Optimus treats system governance not as a passive logging layer, but as an active, objective integrity check on core architectural traits. Phase 1 implements highly targeted, local validation primitives organised across five operational planes: Scope (Atomic), Automation (Automated), Invocation (Triggered), Return (Static), and Activation (Proactive). These gates intercept code generations mid-flight to fix structural deviations before changes lock into local Git trees.

4. Data Governance Plane & Local Storage Boundaries

To support enterprise compliance review, data within the local container is sharply bounded. Unparsed source code is strictly prohibited from entering persistent vector indexes. The storage engine uses standard Redis HASH structures to catalogue structural class signatures, summaries, and relative file paths. Numeric performance histories are channelled separately to a Redis TimeSeries cluster (tracking tokens_input, tokens_output, cost_usd) tagged by unique run_id strings and protected by a hard 30-day retention window policy.

5. Request-Level Cost Attribution

5A. Upstream Aggregator Cost Normalization and Single-Key Model

OPTIMUS_API_KEY remains the only agent-facing credential. It authenticates the agent to the strict-loopback Gateway and is not an upstream vendor key. The Gateway may hold one model aggregator credential plus multiple operator-provisioned, profile-scoped MCP credentials. The architectural invariant is zero upstream credentials in the agent process; Gateway credential count is capability- and profile-dependent.

The developer funds the aggregator account directly. Existing model and typed-tool usage retains the settled GatewayUsage contract. MCP calls use separate attribution-aware MCPUsageRecord records so provider-reported spend, operator-asserted zero marginal cost, and unavailable cost are never conflated.

Approved MCP descriptors are recurring model-input cost. A pre-model admission gate exposes only the operator-selected approved subset, enforces descriptor-count and UTF-8-byte ceilings, and records admitted identities, count, and bytes. Provider-reported model input usage remains the billing authority; Optimus does not estimate per-descriptor tokens. Semantic tool search, automatic per-turn selection, and code mode are deferred to P11-FU-15.

Sampling is closed in v1 because a server could initiate model spend with server-supplied prompt content. A future opening requires pre-model budget reservation, linked provider usage and MCPUsageRecord, and human review before both model dispatch and response return.

Stage	Credential and responsibility
Local agent	OPTIMUS_GATEWAY_URL and OPTIMUS_API_KEY only; zero upstream credentials
Loopback Gateway	Shared-secret auth, profile custody, policy, budget, attribution, and secret isolation
Aggregator	Models, routing, provider-reported billing units and cost_usd
MCP profiles	Operator-provisioned transport, allowlist, binding, and credential references

6. Deterministic Data-Flow Architecture (Phase 1 MVP)

The closed-loop Harness Engineering sequence remains the primary model path:

1. User request enters through the IDE/ACP client and the Optimus ACP server.
2. The agent reads structural memory, applies context optimization, and sends minimal model input to the stateful router.
3. The loopback Gateway delivers the approved model request to the configured aggregator and returns provider-reported usage and USD cost. The agent never selects or contacts a direct provider adapter.
4. Fitness functions validate the result before storage and release.

After the pre-tool decision and before result consumption, an approved namespaced MCP tool follows a separate guarded branch. The agent validates the Plan 6.5 trust record and sends only the profile ID, upstream tool name, arguments, non-secret manifest hash, and profile revision to the Gateway. The Gateway independently validates the active profile, exact binding pair, upstream allowlist, resource limits, and budget policy before contacting the configured stdio or Streamable HTTP server. Tool arguments are the only agent-originated payload sent upstream; system prompts, conversation history, policy text, and approval records never cross. Returned content is untrusted.

Registration and refresh use `server/discover` followed by complete bounded `tools/list` pagination. Cursor loops, malformed cursors, malformed pages, or incomplete pages reject discovery as a whole; a partial tool prefix cannot reach approval or the planner. Transient transport failures retry through the existing policy. A provisioned `page/tool/byte/time` bound yields no manifest and the narrow capacity disposition `mcp.discovery_budget_exceeded`; v1 restarts a complete scan rather than storing cursor checkpoints.

7. Agent Operating Modes & Trust Framework

Plan/Chat remains advisory-only. Agent Mode is write-authorized only after the composite fitness and mutation gates. MCP adds no new agent authority: Plan 6.5 remains authoritative for human approval, descriptor trust, permission scope, and effect class, while the Gateway is authoritative for profile state, credential custody, binding freshness, transport execution, resource limits, and budget admission.

Provisioning and connection lifecycle are separate axes. A profile-scoped HTTP request or stdio child may open only after active-profile admission; opening or closing the connection never activates, re-enables, or changes a profile. Remote HTTP is request-scoped with no protocol session. Stdio reuse is bounded Gateway custody, not MCP session resume. Every stdio profile is Docker contained with a digest-pinned image, no host mounts/devices/socket, and safe `docker run --env NAME` credential projection.

- **PATCH_PROPOSAL:** Token streams over 15 lines that edit existing code assets exclusively without file structural modifications.
- **FILE_MUTATION:** Single-file generation operations, re-writes, or file deletions that require shell checking or tool alignment.
- **MULTI_FILE_CHANGESET:** High-risk operations touching shared/core architecture modules, crossing distinct directory roots, or altering multi-package configuration environments.

8. Tool Governance & Evidence Acquisition

Optimus treats tool use as policy-driven first, model-requested second: the model may ask for evidence, but the harness alone decides whether a tool is allowed, necessary, and cost-justified. This keeps tool invocation aligned with the same cost-control discipline applied to model routing.

Tool Stack (Phase 1)

Tool Class	Examples	Invocation Style
Local repo discovery	rg, file reader, AST parser, dependency graph, git diff	Deterministic, cheap, first-line
Validation gates	Tests, lint, type check, architecture fitness checks, security scan subset	Deterministic after patch generation
Patch workspace	Shadow apply, diff parser, rollback, working-tree apply	Agent mode only
Cost telemetry	RedisTimeSeries, run metadata hash, pricing snapshot	Always-on internal telemetry
Web evidence	Gateway search + extract (Tavily-backed)	Policy-triggered, read-only
Package/version metadata	PyPI, npm, Maven Central, GitHub releases, OSV.dev	Only when dependency/version/security task requires it
External execution	Shell, build, test, package install	Agent mode only, gated

Deterministic Invocation Policy

A ToolInvocationPolicy matrix maps task signals to permitted tool classes, so "search the web" is never a casual model impulse. Representative triggers:

- User explicitly asks for current/latest/external facts → web search.
- Task mentions library/API/framework behaviour not present in the repo → official-doc web search.
- Task touches dependencies → package registry lookup.
- Task touches security/CVEs → OSV or advisory lookup.
- Task asks for a local code change → local repo search first, no web by default.
- Model makes an architectural claim without inspected evidence → file inspection or Assumption Ledger entry.
- Patch generated → shadow apply + fitness gates.

- Fitness gate fails → reflection or targeted local inspection.
- Plan/Chat mode active → read/search tools only, no mutation tools.

Each web-evidence request carries an explicit reason code (e.g. API_DOCS_OUTDATED, CURRENT_FACT, SECURITY_ADVISORY, USER_REQUESTED, PACKAGE_VERSION) plus an allowed-domain list, result cap, and search-depth setting, so external calls remain auditable and cost-bounded rather than ad hoc.

Web evidence acquisition is wrapped behind Optimus-owned typed tools rather than exposing a third-party search API directly to the model. This lets the harness enforce allowed domains, timeouts, retries, result caps, and cost telemetry uniformly. Key rule: local evidence first, external evidence only when policy-triggered, mutation only after mode and fitness gates pass.

9. Adaptive Agent Execution Strategy & Rigor Policy

Optimus applies adaptive execution strategies to balance cost, rigor, and reliability. The agent selects Direct Execution, Plan-and-Execute, ReAct, or Reflection based on task ambiguity, mutation risk, and validation outcomes. Reflection and deep planning are not default behaviours; they are exception-triggered circuit breakers invoked only when risk signals, failed validation, or insufficient evidence require additional rigor.

Low-Cost Hallucination Controls

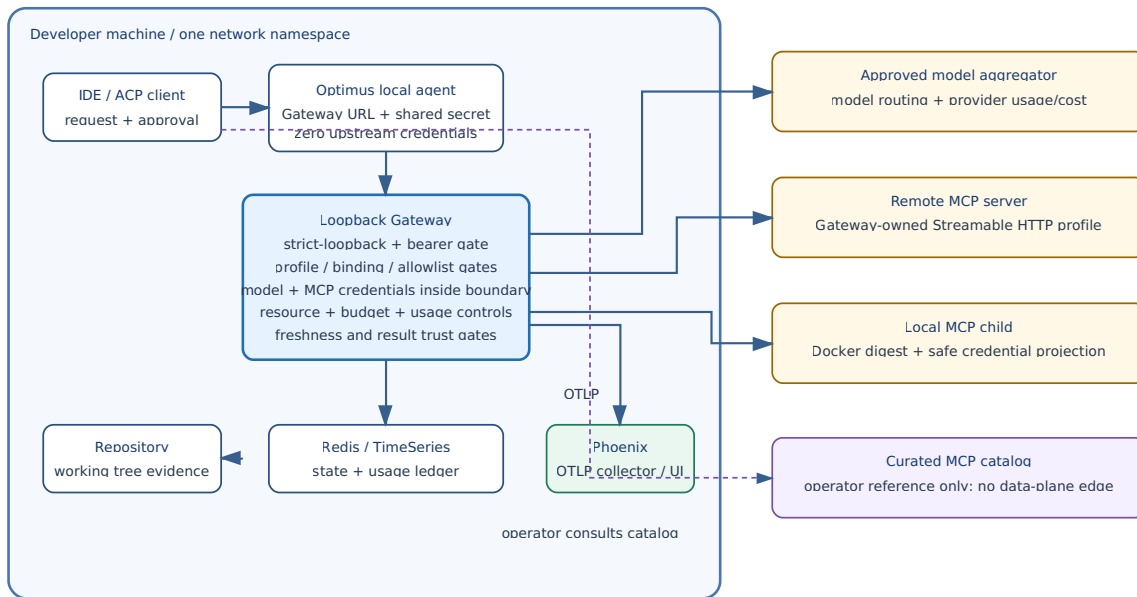
- **Evidence-Bound Decisions:** No architectural claim may be promoted to an implementation decision unless backed by inspected context or recorded as an explicit entry in a machine-readable Assumption Ledger.
- **Rigor Budget Matrix:** Bounded request tiers (LOW, MEDIUM, HIGH) prevent the agent from spending tokens on ceremonial planning loops for simple edits:
 - **LOW:** Cap at 5 tool calls, 0 reflection passes, max 100k context tokens, preferred HAIKU model tier.
 - **MEDIUM:** Cap at 10 tool calls, 1 reflection pass, max 150k context tokens, preferred PRO model tier.
 - **HIGH:** Cap at 15 tool calls, 3 reflection passes, max 250k context tokens, preferred PRO model tier.
- **Trigger-Based Reflection:** Reflection initialises only when validation loops trip a failure signature or code touches shared/core architecture modules, preventing quiet token-doubling on every nominal request.
- **Assumption Ledger:** Bypasses token-heavy text summaries by generating an explicit JSON ledger identifying unverified assumptions, confidence thresholds, and specific validation files needed to clear uncertainty.
- **The Escalation Ladder:** Bounds agent tool invocation paths to escalate strictly through a cost-minimising chain: Provided Context → File Inspection → Plan → Execute → Validate → Targeted Reflection → User Escalation.

10. Architectural Control Flow

This section provides visual reference diagrams for the three primary control flow concerns in Optimus: where the system boundaries sit, how the governance loop operates end-to-end, and

10.A System Context Diagram

The IDE, local agent, loopback Gateway, Redis, repository, and optional Phoenix collector are inside one developer environment and network namespace. The agent authenticates with the local shared secret and cannot read Gateway-owned model or MCP credentials. Both Gateway and agent trust gates are required before an MCP call.



System context: Gateway-owned model and MCP edges

The Gateway alone may contact the approved model aggregator, package registries, OSV, configured OTLP collector, remote Streamable HTTP MCP servers, and local Docker-contained stdio MCP children. An operator may consult a curated MCP catalog before provisioning, but the catalog is untrusted metadata: it has no registry-to-agent, registry-to-model, registry-to-Gateway-data-plane, install, connect, or activation edge. An agent in WSL2 cannot reach a Windows-host Gateway through loopback; Phase 1 requires both processes in the same network namespace.

Figure 10.A - Local-first system context. MCP servers are reachable only through the strict-loopback Gateway after both agent and Gateway trust checks; no upstream MCP credential enters the agent process.

B. Governance Loop Diagram

Every user request traverses the full governance loop. The loop enforces that no model output reaches the working tree without passing through both the feedforward constraint layer and the feedback fitness gate layer.

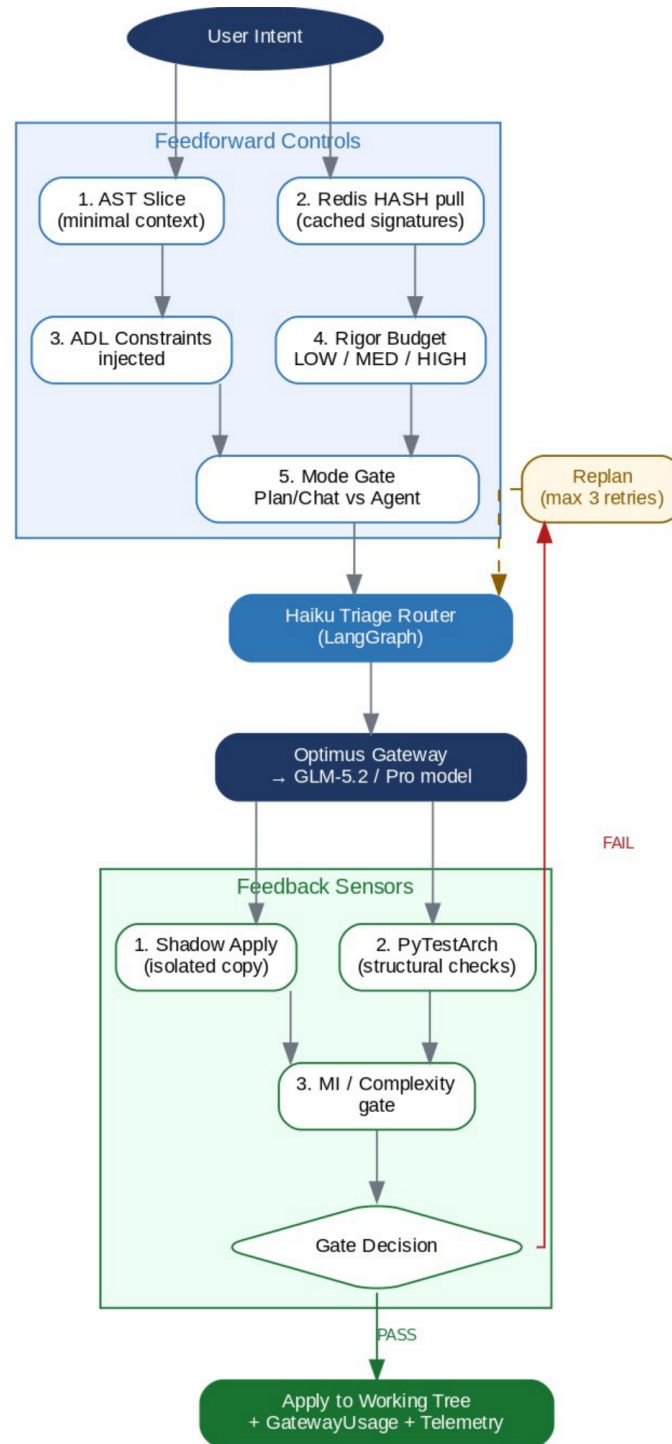


Figure 10.B · Governance Loop — feedforward controls, triage routing, feedback sensors, mutation gate.

10.C Phase Evolution Diagram

Optimus is designed as a three-phase architecture. Phase 1 is mandatory before later phases.

PHASE 1 -> Python Local Agent + Optimus Gateway Current - Mandatory before Phase 2

Release Gate: the agent runs with only `OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY`; no model, search, MCP, OTel, or other upstream credential is resolvable in the agent process. The Gateway receives only the approved credentials and profile records required by its enabled capabilities, with MCP secrets isolated per profile.

PHASE 2 -> Rust Core Rewrite Trigger: Python performance ceiling or team scaling

PHASE 3 -> Agentic Mesh Topology Trigger: multi-repo or multi-team deployment

10.D Where Cost Control Happens

Control point	Mechanism	What it prevents
Pre-prompt	AST slicing, cache anchoring, ADL constraints	Unnecessary model input
MCP context admission	Operator-selected descriptor subset, count/UTF-8-byte ceilings, admitted identity record	Descriptor-context cost and context bloat
Router and rigor budget	Cheapest sufficient model plus bounded calls/reflection	Ceremonial spend
Post-call attribution	GatewayUsage, MCPUsageRecord, EvidenceLedger, RedisTimeSeries	Silent accumulation and unclassified MCP cost

10.E Where Hallucination Control Happens

Context, evidence, typed-tool policy, fitness functions, and bounded reflection remain a defence-in- depth stack. MCP descriptors, arguments, tool results, server identity, instructions, annotations, resource links, and embedded resources are untrusted input and cannot modify policy, approve tools, trigger a fetch, execute code, or become trusted manifest content.

11. Optimus Gateway - Phase 1 Local Process Boundary

The Optimus Gateway is a deterministic strict-loopback process run alongside the local agent. It is not a hosted service, tenant control plane, subscription product, or central wallet.

Gateway responsibilities

- Authenticate the agent and keep all model, OTel, and profile-scoped MCP credentials inside the Gateway boundary.
- Serve typed MCP discovery and call routes; inside the Gateway use `server/discover`, `tools/list`, and `tools/call` with tools-only filtering.
- Enforce profile revision, manifest hash, exact binding pair, upstream allowlist, freshness, pagination, resource, and budget checks before transport execution.
- Use per-request metadata for remote Streamable HTTP 2026-07-28; do not use a legacy remote initialize handshake, client ping, protocol session, or standalone GET stream.
- For mandatory Docker-contained stdio, probe first and negotiate modern or legacy tools-only behavior. The remote HTTP floor deliberately narrows the official Go SDK v1.7.0 default, which may negotiate down to 2025-11-25 or earlier.
- Attribute MCP calls to `MCPUsageRecord`, persist usage before releasing accepted results, and tear down profile-scoped connections without changing provisioning state.

The 2026-07-28 floor is deliberately breaking for remote HTTP credential profiles. A server that cannot establish the required discovery/version/tools contract receives the narrow `mcp.protocol_version_unsupported` disposition and never falls back. Context7 is the named remote-compatibility dependency: its public Streamable HTTP configuration is the motivating case, not proof of deployed protocol support. Reachability remains unconfirmed until an authenticated, Gateway-originated probe of the configured endpoint proves the exact version and tools capability.

The immutable snapshot `f817239f4d6b1efff2c4dfc2f7af85c985d73076` is the frozen wire-content source under `schema/draft/`, not final per-version specification publication. Go SDK v1.7.0 is the support citation. External MCP logging is closed in v1 and never feeds or changes Optimus's append-only audit logging or telemetry.

Agent process environment	Gateway-only configuration
<code>OPTIMUS_GATEWAY_URL</code> , <code>OPTIMUS_API_KEY</code>	model aggregator credential and profile credential references
no upstream credentials	profile transport, revision, binding, allowlist, and freshness
no secret-derived identifier	discovery page/tool/descriptor-byte/time and context ceilings

Split-authority residual

The agent remains authoritative for human approval, descriptor trust, permission scope, and effect class. The Gateway is authoritative for profile state, credential custody, upstream-name allowlists, binding freshness, transport execution, resource limits, and budget admission. A direct shared-secret caller can bypass agent-only scope/effect checks but cannot exceed the operator-provisioned Gateway allowlist. A recoverable refresh failure marks the last bound manifest stale rather than denying it; detected drift remains a denial. This is accepted only for Phase 1's strict-loopback, single-operator deployment.

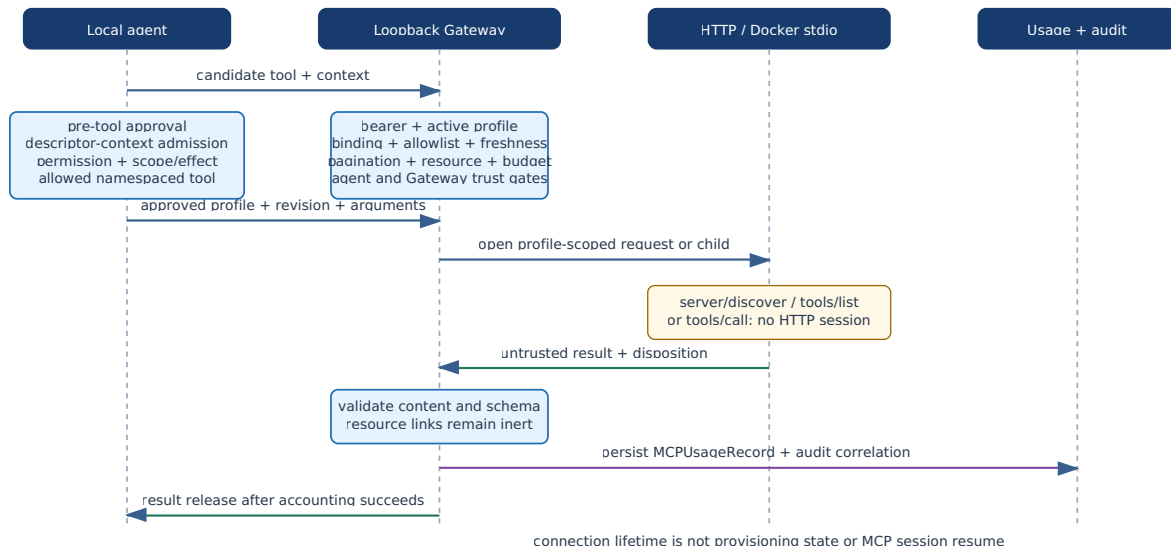
Architecture positions

- Registry catalog metadata is untrusted operator input and cannot install, connect, or activate a profile; the catalog is not a runtime data-plane authority.
- OAuth is static-credential-only in v1. The future binding discriminator distinguishes automatic same-grant refresh from reapproval-triggering rotation; a binding change mints a new revision.
- `input_required` remains closed at method, result-type, and content boundaries until the attributed operator UI, schema validation, and bounded continuation conditions exist. Elicitation is therefore denied call-scoped in v1 and never becomes a planner question or profile transition.
- Normative MCP controls extend Plan 6.5 only. Generalized security observations remain reference-only and create no independent Test Strategy obligation.

Architecture reference and ownership

General observation	Voice / owner
OWASP LLM01 tools and retrieval; LLM02 output handling; LLM03 configuration; LLM05 packages and plugins; LLM06 model/tool output; LLM07 permissions, autonomy, and spend; LLM10 hidden context, retries, long-lived work, and resource use	REFERENCE – Cross-cutting (non-normative guidance; not a Test Strategy obligation)

11.1 Gateway request / response sequence



Request sequence from agent to Gateway-owned model and MCP transports

For a namespaced MCP call, the agent performs pre-tool approval and sends the profile ID, upstream tool name, arguments, non-secret manifest hash, and profile revision. The Gateway authenticates the bearer, checks active profile state, binding pair, allowlist, freshness, resource limits, and budget, then opens a profile-scoped HTTP request or Docker-contained stdio child. The connection lifetime begins after admission and never changes provisioning state.

The Gateway validates returned content as untrusted data, persists the MCP-specific usage record, and releases the result only after required accounting succeeds. A persistence failure withholds the result and retries persistence only; it never redispatches the upstream tool call. HTTP has no MCP protocol session, and bounded stdio reuse is not session resume. The agent has no direct MCP edge and no upstream MCP credential.

11A. OpenTelemetry Trace Observability

OpenTelemetry spans and OTLP remain vendor-neutral across planning, Gateway calls, MCP invocation, validation, retries, and response generation. The Gateway validates and redacts structured trace ingress before export. MCP external logging is deprecated and closed; it is distinct from the append-only Optimus audit log and telemetry.

12. Testing and quality gates

Concern	Required evidence
Protocol and transport	Real remote HTTP 2026-07-28 method-set evidence with no legacy handshake/session/ping; Docker-contained stdio discovery-first modern/legacy negotiation
Discovery	Complete multi-page tools/list; cursor integrity distinct from transient/capacity outcomes; no partial manifest
Trust and custody	Direct-route allowlist denial, profile-change/revision reapproval, credential-isolation scans, platform process-limit evidence, and strict-loopback namespace evidence
Accounting	Descriptor-context ceilings, settled/explicit_zero/unavailable MCP attribution, persistence-before-release, and no redispach after accounting failure
Safety and capability	Elicitation triple denial is call-scoped; sampling causes no model call or spend in v1; capability absence never disables the whole profile
Provenance and ownership	REFERENCE – Cross-cutting versus NORMATIVE – P11-FEAT-GATEWAY-MCP voice classification, redline traceability, Context7 authenticated real-server probe, and external logging denial with Optimus audit unchanged

Test Strategy v1.6 is authoritative for executable evidence. No route, profile, transport, ledger, or sandbox claim is complete until its named evidence passes.

13. Agent Execution Safety & Guardrails

Agent execution safety is enforced through a layered guardrail model that wraps every tool call between the moment the agent decides to act and the moment the action takes effect: **mode constraints** (§7), a **permission policy** (allow/deny lists with a human-approval path), a **pre-tool guard** that deterministically validates shell, file, MCP, and network calls, **sandboxed execution**, **post-tool audit** against the Gateway cost and trace ledgers (§5A, §11, §11A), and **commit/CI gates** (§12). The same plane governs two execution patterns — **bounded goal-driven agent loops** and **curated workflow skills** — so that long-running or procedure-driven work inherits the identical constraints and the same Gateway budget policy.

The premise is not that the agent is malicious but that it is a reward-hacky problem solver: under pressure to reach "done" it will reach for `chmod 777`, `curl ... | sh`, force-push to main, or commenting out a failing assertion. Guardrails remove those shortcuts from its reach. Most of the model is deterministic and zero-token; a cheap, budgeted classifier routed through the Optimus Gateway is consulted only for borderline cases, never on the hot path of an already-allowed or already-denied call.

The detailed policy model, component contracts, and control-flow diagram are maintained in the companion specification **Agent Execution Guardrails & Workflow Strategy v1.0** (control-plane diagram in its §1; LLD component contracts in its §10; test categories in its §11). The corresponding implementation contracts are specified in **LLD §12** and the test categories in **Test Strategy §14**. This section is a cross-cutting reference only.